

Semana 2 Búsqueda Informada y ambientes complejos

ISIS-1611: Inteligencia Artificial

Carla González

10 de junio de 2026

1. Búsqueda Informada Avanzada

1.1. Búsqueda con Memoria Acotada

1.1.1. Iterative Deepening A* (IDA*)

Versión de A* que usa IDDFS con el valor de corte $f(n) = g(n) + h(n)$.

- Espacio: $\mathcal{O}(bd)$
- Completo y óptimo (con h admisible)

1.1.2. Recursive Best-First Search (RBFS)

Mantiene el mejor valor alternativo f en cada nivel de recursión.

- Espacio: $\mathcal{O}(bd)$
- Puede reexpandir nodos (regenera sub-árboles olvidados)

1.1.3. SMA* (Simplified Memory-Bounded A*)

Usa toda la memoria disponible; descarta el nodo con mayor f cuando la memoria está llena.

1.2. Heurísticas: Dominancia y Creación

Definición 1.1 (Dominancia). h_2 domina a h_1 si $h_2(n) \geq h_1(n)$ para todo n y ambas son admisibles. Una heurística dominante expande menos nodos.

Formas de construir heurísticas admisibles:

1. **Relajación del problema:** eliminar restricciones del problema original.
2. **Bases de datos de patrones:** precomputar costos exactos de subproblemas.
3. **Máximo de heurísticas:** $h(n) = \max(h_1(n), h_2(n))$

2. Búsqueda en Ambientes Complejos

2.1. Búsqueda Local

A diferencia de la búsqueda sistemática, la búsqueda local opera sobre un **estado actual** sin mantener el camino recorrido. Útil cuando el objetivo es el estado mismo (no el camino).

2.2. Hill Climbing (Ascenso de Colina)

Problemas:

- Máximos locales (no globales)
- Mesetas (*plateaus*) y crestas

Variantes: Hill climbing estocástico, reinicio aleatorio, *simulated annealing*.

2.3. Simulated Annealing

Permite movimientos malos con probabilidad $e^{\Delta E/T}$ donde T decrece con el tiempo. Garantiza convergencia al óptimo global si T baja suficientemente despacio.

2.4. Algoritmos Genéticos

Inspirados en la evolución biológica. Operan sobre una **población** de individuos (estados codificados como cadenas).

Operadores:

1. **Selección:** individuos con mayor aptitud (*fitness*) tienen más probabilidad de reproducirse.
2. **Cruzamiento** (*crossover*): combina dos individuos para producir descendencia.
3. **Mutación:** cambia aleatoriamente genes con baja probabilidad.

3. Ejercicios de Búsqueda

Nota

Esta clase es de ejercicios y repaso para el **Examen 1**. Ver la *Guía de Ejercicios: Búsqueda Desinformada e Informada*.

3.1. Guía de Ejercicios — Resumen de Bloques

3.1.1. Bloque 1: Agentes Inteligentes

Caracterizar escenarios con PEAS, identificar tipo de agente y entorno.

3.1.2. Bloque 2: Formulación de Problemas

Definir estado inicial, modelo de transición, costo y test de meta para problemas concretos.

3.1.3. Bloque 3: Búsqueda No Informada

Aplicar BFS, DFS y UCS sobre grafos; analizar completitud y optimalidad.

3.1.4. Bloque 4: Búsqueda Informada y Heurísticas

Aplicar Greedy, A*, RBFS, SMA*; verificar admisibilidad y consistencia de heurísticas.

Ejercicio 3.1. Dado el grafo del Bloque 3 (Guía), aplica BFS, DFS y UCS. Indica el orden de expansión y el camino solución encontrado.

Solución:

4. Heurísticas

Es una función que estima que tan cerca está un estado a su meta, guía para el camino con mayor promesa. Las heurísticas es que no sobrestimen los costos. Si dos estados tienen una evaluación heurística similar -> preferiblemente examinar estado mas cercano a la raíz en grafo de estados.

Definición 4.1. Una **heurística** es una función que estima que tan cerca está un estado n de la meta, diseñada específicamente para un problema de búsqueda particular.

$$h : \text{Estados} \rightarrow \mathbb{R}_{\geq 0}$$

En el contexto de búsqueda informada, $h(n)$ representa el **costo estimado** del camino mas barato desde el estado n hasta la meta mas cercana. Por convención, $h(\text{meta}) = 0$

En términos generales, una heurística es el estudio de metodos y reglas de descubrimiento e invención. En IA, nos interesa como *aproximación informada* que:

- usa conocimiento del dominio para guiar la búsqueda.
- descarta/remueve posibilidades de solución poco prometedoras
- no garantiza exactitud, pero si eficiencia práctica
- **distancia en linea recta:** útil en mapas de carreteras. $h(n)$ = distancia euclideana de n a la meta.
- **distancia manhattan:** útil en grillas. $h(n) = \sum_i |x_i - x_{\text{meta}}| + |y_i - y_{\text{meta}}|$
- **celdas fuera de lugar:** útil en el 8-puzzle. $h(n)$ = número de piezas no en su posición goal.

4.1. Repaso algoritmos de búsqueda informada

Los algoritmos estudiados en la clase anterior usan $h(n)$ de distintas formas:

Algoritmo	Función de evaluación	Estrategia
Greedy Best-First	$f(n) = h(n)$	Minimizar costo restante
A*	$f(n) = g(n) + h(n)$	Minimizar costo total
A* con peso	$f(n) = g(n) + w \cdot h(n)$	Sesgar hacia la meta

donde $g(n)$ es el costo acumulado desde el inicio hasta n (*backward*), y $h(n)$ es la estimación del costo restante (*forward*).

5. Admisibilidad, Monotonicidad, Informedness(Desigualdad triangular)

5.1. Admisibilidad

Definición 5.1 (Admisibilidad). Una heurística h es **admisible** (u *optimista*) si nunca sobreestima el costo real para llegar a la meta.

$$h(n) \leq h^*(n), \quad \forall n$$

donde $h^*(n)$ es el costo verdadero del camino optimo desde n hasta la meta.

Consecuencia clave: si h es admisible, A* sobre arbol de busqueda es *optimo*: nunca descarta la solución optima porque nunca penaliza en exceso ningun nodo.

Ejemplo 5.1 (Admisibilidad en BFS). BFS usa $f(n) = g(n)$ (equivalente a $h(n) = 0$ para todo n). Como $0 \leq h^*(n)$ siempre, $h \equiv 0$ es admisible (aunque trivial e ineficiente).

Teorema 5.1 (heurísticas inadmisibles). Una heurística **inadmisible** (*pesimista*) puede hacer que A* descarte nodos con soluciones óptimas atrapados en la frontera, rompiendo la garantía de optimalidad.

5.2. Monotonicidad (Consistencia)

Definición 5.2 (Monotonicidad/consistencia). Una heurística h es **monotonica** o consistente si para todo nodo n_i y cualquier sucesor n_j :

$$h(n_i) - h(n_j) \leq \text{cost}(n_i, n_j)$$

equivalentemente,

$$h(n_i) \leq \text{cost}(n_i, n_j) + h(n_j)$$

Además, se requiere $h(\text{meta}) = 0$.

Esta condición es mas fuerte que la admisibilidad. Su intuición geometrica es que la heuristica no puede "saltar" hacia atrás mas que el costo real del arco.

consecuencias de la consistencia:

- El valor de $f(n) = g(n) + h(n)$ es **no decreciente** a lo largo de cualquier camino: $f(n_j) \geq f(n_i)$ siempre que n_j sea sucesor de n_i .
- En A* con grafo de búsqueda (eliminación de repetidos), una heurística consistente garantiza que el primer camino encontrado a cada nodo es el óptimo. Por lo tanto, **no es necesario revisitar nodos**, lo que reduce la complejidad temporal.

La consistencia $\Rightarrow$ admisibilidad; la implicación no es recíproca en general.

5.3. desigualdad del triangulo

La consistencia es esencialmente la *desigualdad del triangulo* aplicada a las estimaciones heurísticas:

$$d(A, C) \leq d(A, B) + d(B, C)$$

6. Ambientes Complejos

En esta sección se involucra la **relajación de restricciones**, donde hay presentes *problemas, algoritmos, estrategias y problemas no determinísticos*

Definición 6.1 (Problemas de Búsqueda). 1. Completamente observables

2. determinísticos
3. Estáticos
4. ambientes conocidos

Definición 6.2 (Problemas discretos de hallar un buen estado). 1. Búsqueda hill-climbing

2. simulated annealing
3. local beam search
4. evolutionary algorithms

Hasta ahora los algoritmos de búsqueda (BFS, DFS, UCS, A*) asumen ambientes **completamente observables, determinísticos, estáticos y conocidos**, y su objetivo es encontrar un *camino* desde el estado inicial hasta la meta.

Sin embargo, hay clases de problemas donde lo que importa no es el camino sino el **estado final** en si mismo. esta sección estudia tres familias de técnicas para esos casos:

1. **Busqueda local en espacios discretos:** Hill-climbing, simulated annealing, local beam search y algoritmos evolutivos.
2. **Busqueda local en espacios continuos:** discretización, gradiente empirico, newton-raphson y programacion lineal
3. **Busqueda con acciones no deterministas:** arboles AND-OR y planes condicionales

En busqueda local **no se guarda el camino recorrido**; el estado actual es todo lo que importa. Esto reduce dramaticamente el uso de memoria, al costo de no garantizar optimalidad global.

7. Busqueda Local y Problemas de optimización

Definición 7.1 (busqueda local). Clase de algoritmos que dado un estado actual, generan sus vecinos y se mueven al mejor de ellos segun una **funcion objetivo** $f : \mathcal{S} \rightarrow \mathbb{R}$. no mantienen frontera ni arbol de busqueda completo.

El espacio de busqueda se puede visualizar como un *paisaje de estados*: los estados son posiciones horizontales y su valor f es la altura. El objetivo tipico es encontrar el **maximo global** o minimo dependiendo de la formulacion.

7.1. Hill-Climbing

Definición 7.2. Mantiene registro de un unico estado actual, y en cada iteración, se desplaza al vecino con **mayor valor** de f . termina cuando ningun vecino supera al estado actual. Tambien llamado *steepest-ascent* o *busqueda local voraz*

Ejemplo 7.1 (Hill-Climbing clasico). 1. Estado actual $\leftarrow$ estado inicial

2. Generar todos los vecinos del estado actual
3. Si algun vecino tiene f mayor: moverse al mejor vecino y volver a 2
4. Si ningun vecino mejora: **terminar** (maximo local)

7.2. Variantes

- Steepest-ascent (determinista): evalúa *todos* los vecinos y toma el mejor. Garantiza el mayor avance en cada paso, pero es costoso.
- Stochastic Hill-Climbing: elige al azar entre los vecinos que mejoran el estado actual. Menos cómputo por paso; puede escapar de algunos mesetas.
- First-choice hill-climbing: genera y evalúa vecinos en algún orden, toma el *primero* que mejore. útil cuando el numero de vecinos es muy grande. Ventaja: menos cómputo, desventaja: depende del orden de generación
- Random-restart hill-climbing: al atascarse en un maximo local, elige un nuevo estado inicial aleatorio y repite. Aumenta la probabilidad de encontrar el optimo global, a costa de mas tiempo

7.3. Limitaciones

Teorema 7.1 (Problemas de hill-climbing). *Hill-climbing puede quedar atrapado en:*

- **Maximos locales:** pico mas alto que sus vecinos, pero no el maximo global.
- **Mesetas:** región plana donde todos los vecinos tienen el mismo valor
- **Crestas:** serie de maximos locales separados por descensos, dificiles de cruzar

*Nunca realiza movimientos que **desciendan** la funcion objetivo.*

Ejemplo 7.2 (Problema de las 8 reinas). **Estado:** Posición de las 8 reinas, una por columna. h = numero de pares de reinas en conflicto (queremos minimizar) La función objetivo es $f = 28 - h$ (queremos maximizar) Con el tablero [6, 3, 8, 5, 5, 2, 1, 4] y First-Choice: se prueban cambios columna a columna hasta encontrar el primero con menor h . Con Steepest-Ascent a partir de un estado aleatorio, hill-climbing resuelve el 97 % de los casos, pero el 3 % restante requiere Random-Restart

7.4. Simulated Annealing (recorrido simulado)

Definición 7.3 (Simulated Annealing). Variante de Hill-Climbing que permite movimientos que *empeoran* la funcion objetivo con una probabilidad que decrece con el tiempo. La probabilidad de aceptar un empeoramiento $\Delta E < 0$ es

$$P = e^{\Delta E/T},$$

donde T es la **temperatura** actual del sistema.

1. Estado actual $\leftarrow$ estado inicial.
2. Para cada tiempo $t = 1, 2, \dots$:
 - a) $T \leftarrow \text{Schedule}(t)$. Si $T = 0$, **terminar**.
 - b) Elegir un vecino v al azar.
 - c) $\Delta E \leftarrow f(v) - f(\text{actual})$.
 - d) Si $\Delta E > 0$: moverse a v (mejora).
 - e) Si $\Delta E \leq 0$: moverse a v con probabilidad $e^{\Delta E/T}$.

La intuición es la del proceso físico de *recocido*: a temperatura alta se permite mucha exploración (movimientos aleatorios); al enfriarse el sistema, el algoritmo se vuelve mas exigente y converge a un minimo(maximo).

Teorema 7.2 (Comportamiento según temperatura). ▪ T alta $\Rightarrow$ **exploracion**: se aceptan muchos movimiento que empeoran

- T baja $\Rightarrow$ **explotación**: comportamiento similar a Hill-Climbing
- Si T decrece lo suficientemente lento, Simulated Annealing encuentra el óptimo global con probabilidad 1 (en el limite)

Ejemplo 7.3 (Traza de simulated annealing).

Paso	h (conflictos)	T
0	5	100
1	6	100
2	4	50
3	4	20
4	3	5
5	3	1
6	2	0.1
7	0	0.01

El paso 1 **empeoró** ($h : 5 \rightarrow 6$) pero fue aceptado gracias a la temperatura alta. A partir del paso 5, la temperatura es tan baja que el algoritmo prácticamente no acepta empeoramientos: ya se comporta como hill-climbing

7.5. Local Beam Search

Definición 7.4 (Local Beam search). Extensión de hill-climbing que mantiene k **estados en paralelo** en lugar de uno solo. En cada iteración se generan los sucesores de los k estados y se conservan los k mejores de toda la lista combinada.

1. inicializar con k estados aleatorios
2. generar todos los sucesores de los k estados
3. si algún sucesor es meta: **terminar** (éxito)
4. seleccionar los k mejores sucesores (de la lista completa) y repetir desde 2

La diferencia clave con Beam Search clásico: Beam Search es búsqueda en árbol por niveles con poda (mantiene k nodos por nivel); Local Beam Search es búsqueda local con múltiples estados en paralelo *sin* árbol explícito.

Problema frecuente es los k estados tienden a *converger* al mismo máximo local porque comparten información. la variante **estocástica** selecciona los k siguientes estados con probabilidad proporcional a su f , lo que mantiene mayor diversidad.

7.6. Algoritmos evolutivos

Definición 7.5 (algoritmos evolutivos). Familia de métodos de búsqueda local que operan sobre una **población de k soluciones** y las mejoran mediante operaciones inspiradas en la evolución biológica, selección y recombinación (crossover) y mutación.

La tabla siguiente compara las cuatro familias vistas:

Algoritmo	Soluciones	Mecanismo extra
Hill-Climbing	1	—
Simulated Annealing	1	Azar (temperatura)
Local Beam Search	k	—
Algoritmos Evolutivos	k	Recombinación + Mutación

7.6.1. Ciclo de vida de un algoritmo evolutivo

1. **Generación 0**: crear k soluciones (individuos) aleatorias.
2. **Evaluar**: calcular el *fitness* de cada individuo.

3. **Seleccionar**: elegir los mejores individuos como padres (selección proporcional al fitness, torneo, etc.).
4. **Combinar (Crossover)**: recombinar pares de padres para generar hijos mezclando sus genes.
5. **Mutar**: aplicar cambios aleatorios pequeños a algunos hijos.
6. **Repetir** desde el paso 2 con la nueva generación.

Ejemplo 7.4 (8 Reinas con Algoritmo Evolutivo). **Representación**: individuo = [4, 6, 1, 5, 2, 8, 3, 7] (posición de cada reina en su columna).

Fitness: $f = 28 - (\text{pares de reinas en conflicto})$. El máximo posible es 28 (ningún conflicto = solución).

Crossover con punto de corte en la posición 4:

Padre 1: [4, 6, 1, 5 | 2, 8, 3, 7]

Padre 2: [2, 5, 7, 4 | 8, 1, 6, 3]

Hijo 1: [4, 6, 1, 5, 8, 1, 6, 3]

Hijo 2: [2, 5, 7, 4, 2, 8, 3, 7]

Mutación: cambiar un gen al azar; por ejemplo: [4, 6, 1, 5, 7, 1, 6, 4].

7.7. Parametros y variaciones

- **Tamaño de la población k** : poblaciones grandes exploran más pero son más costosas.
- **Representación del individuo**: binaria, entera, permutaciones, árboles, etc.
- **Número de padres**: normalmente 2; en algunos esquemas se usan más.
- **Proceso de selección**: proporcional al fitness, selección por torneo, elitismo.
- **Punto de crossover**: fijo, aleatorio, multipunto.
- **Schema**: substring donde algunas posiciones son *comodín* (ej. 246*****). Los algoritmos evolutivos explotan implícitamente schemas asociados a componentes útiles de la solución.